



Instituto Tecnológico y de Estudios Superiores de Monterrey

Asignatura: Procesamiento del Lenguaje Natural

Proyecto Final: Sistema Inteligente para búsqueda de talento

GitHub: [ZaifenKL/TalentMatcher-RAG at Path_Fix](https://github.com/ZaifenKL/TalentMatcher-RAG-at-Path-Fix)

Zaifen King Loeza A01797887

Docente: Luis Eduardo Falcon Morales

Fecha: 28/06/2026

Contenido

1.0 Introducción	4
1.1.0 Objetivo del Sistema.....	5
2.0 Vacante seleccionada.....	5
2.1.0 Empresa	5
2.1.1 Nombre de la Vacante	6
2.1.2 Descripción del Puesto.....	6
2.1.3 Competencias Requeridas	6
2.1.4 Responsabilidades Principales del rol.....	7
2.1.4 Requisitos Técnicos	7
3.0 Generación de CVs sintéticos	8
3.1.0 Importancia de no incluir datos sensibles	9
4.0 Marco Teórico.....	10
5.0 Arquitectura General del Sistema	11
6.0 Implementación	13
6.1.0 Extracción y Procesamiento de documentos	13
6.1.1 Extracción mediante OCR y parser de PDF	13
6.1.2 Segmentación y preparación del Corpus	14
6.1.2 Generación de Embeddings	15
6.1.3 Construcción del Vector Store	16
6.1.4 Recuperación Semántica y Ranking de CVs	17
6.1.5 Implementación LLM	17
6.1.6 Core.....	19
7.0 Evaluación del Prototipo	20
7.1.0 Prueba de Ruido: Evaluación del comportamiento del sistema ante consultas irrelevantes	21
7.1.1 Modificaciones al sistema	23
7.1.2 Resultados.....	26
7.1.3 Prueba de rol específico y general.....	26
7.1.4 Evaluación del sistema contra la vacante seleccionada.....	29
7.1.5 Evaluación de compatibilidad.....	29
8.0 Limitaciones del sistema.	32
9.0 Impacto y Aplicaciones Potenciales	33

9.1.1 Aplicaciones en Gestión del Conocimiento Empresarial	34
10. Trabajo Futuro	34
11. Conclusiones	35
Referencias.....	35

1.0 Introducción

El procesamiento de lenguaje natural (NLP) se ha consolidado como una de las tecnologías con mayor impacto estratégico para organizaciones que buscan convertir grandes volúmenes de texto no estructurado en representaciones útiles para el análisis y la toma de decisiones. En un entorno donde las empresas manejan documentos como currículums, descripciones de puestos, reportes técnicos o bases de conocimiento internas, la capacidad de interpretar, vectorizar y recuperar información semánticamente relevante se vuelve un diferenciador competitivo clave.

Este proyecto implementa un pipeline completo de NLP orientado a resolver un problema concreto: la búsqueda inteligente y el emparejamiento semántico de perfiles profesionales. A diferencia de los sistemas tradicionales basados en coincidencias literales de palabras clave, este enfoque utiliza técnicas modernas como embeddings de alta dimensión, bases vectoriales y modelos de recuperación aumentada (RAG) para comprender el significado profundo del texto y establecer similitudes reales entre candidatos y requerimientos laborales.

La arquitectura construida incorpora todos los componentes clave de un pipeline moderno de procesamiento de lenguaje natural:

- Extracción y normalización de texto desde documentos PDF e imágenes.
- **Limpieza avanzada** para corregir ruido proveniente de la extracción en PDF y del OCR en imágenes, garantizando un texto uniforme y procesable.
- **Segmentación inteligente (chunking)** que preserva coherencia semántica y evita cortes de tokens.
- **Generación de embeddings**, optimizados para español y dominios técnicos.
- **Construcción de un vector store persistente** mediante técnicas de indexación eficiente.
- **Implementación de búsqueda semántica** para recuperar los fragmentos más relevantes ante una consulta.
- **Integración de un pipeline RAG**, donde un modelo de lenguaje genera respuestas fundamentadas en evidencia real proveniente del vector store.

El proyecto entrega un pipeline integral y funcional que sienta las bases para una solución escalable, capaz de vectorizar y recuperar información con alta precisión en escenarios reales. Este tipo de sistemas contribuye a agilizar procesos que dependen del análisis

documental, favoreciendo una identificación más precisa de patrones, similitudes y contenidos relevantes dentro de grandes volúmenes de información. En contextos donde es necesario comparar perfiles, requisitos o documentos de forma consistente, este enfoque aporta claridad, reduce tiempos asociados al reclutamiento mediante búsquedas más precisas, aumenta la eficiencia operativa al automatizar tareas manuales de lectura y clasificación, y facilita decisiones mejor fundamentadas.

1.1.0 Objetivo del Sistema

El objetivo del sistema es desarrollar una solución inteligente capaz de analizar, comparar y jerarquizar perfiles profesionales mediante técnicas avanzadas de Procesamiento de Lenguaje Natural (NLP), embeddings semánticos y modelos de lenguaje de gran escala (LLMs). El sistema busca demostrar la integración de arquitecturas multimodales y métodos de recuperación semántica para resolver un problema complejo de clasificación y recomendación de talento. Desde una perspectiva de negocio, la herramienta está diseñada para reducir significativamente los tiempos de preselección, mejorar la calidad del filtrado inicial y ofrecer recomendaciones justificadas que apoyen la toma de decisiones en procesos de reclutamiento técnico. El sistema no sustituye al reclutador humano, sino que actúa como un asistente especializado que incrementa la eficiencia operativa, disminuye costos asociados al análisis manual y fortalece la objetividad en la evaluación de candidatos.

Para validar la efectividad del sistema en un escenario realista y alineado con las necesidades actuales del mercado laboral, se seleccionó una vacante especializada en ingeniería de software aplicada a inteligencia artificial. Esta vacante sirve como caso de estudio para evaluar la capacidad del sistema de identificar talento técnico avanzado, analizar competencias específicas y generar recomendaciones fundamentadas. A continuación, se presenta la descripción detallada de la vacante seleccionada y la justificación de su elección.

2.0 Vacante seleccionada

2.1.0 Empresa

CRH Talento de IT es una firma especializada en la búsqueda y selección de consultores en Tecnologías de Información. Su enfoque se centra en proveer talento altamente calificado para organizaciones que requieren perfiles técnicos avanzados, particularmente en áreas de ingeniería de software, inteligencia artificial y automatización. La empresa opera bajo un modelo de consultoría que prioriza la calidad del talento y la alineación estratégica con las necesidades del cliente.

2.1.1 Nombre de la Vacante

Dentro de los perfiles abiertos en su portal de empleo se seleccionó la vacante de: Software Engineer AI – Artificial Intelligence.

Esta vacante fue seleccionada porque representa un rol altamente especializado en el desarrollo de sistemas de inteligencia artificial aplicada, alineado directamente con los objetivos académicos de la actividad y con las tendencias actuales del mercado laboral. El puesto exige competencias avanzadas en GenAI, agentes autónomos y automatización inteligente, lo que permite evaluar de manera precisa la capacidad del sistema para identificar talento técnico de alto nivel. Además, la vacante presenta requisitos detallados y específicos, lo que facilita la construcción de una base de conocimiento rica y adecuada para realizar búsquedas semánticas, comparaciones entre perfiles y análisis de compatibilidad. Su complejidad técnica convierte esta vacante en un caso ideal para validar la efectividad del sistema inteligente desarrollado.

2.1.2 Descripción del Puesto

La vacante busca un ingeniero de software con experiencia sólida en Python, Java y sistemas de inteligencia artificial generativa (GenAI). El rol implica diseñar, construir y desplegar sistemas de agentes autónomos basados en LLMs, así como desarrollar herramientas de automatización que optimicen flujos de trabajo de ingeniería. Se trata de una posición estratégica que impacta directamente en la productividad de los equipos de desarrollo, la calidad del software y la capacidad de la organización para adoptar tecnologías emergentes de IA a escala.

2.1.3 Competencias Requeridas

- ✓ Dominio avanzado de Python y experiencia con Java.
- ✓ Conocimiento profundo de frameworks de agentes como ADK o LangGraph.
- ✓ Capacidad para diseñar arquitecturas limpias aplicadas a sistemas de IA.
- ✓ Habilidades en context engineering, memoria semántica y multi-hop reasoning.
- ✓ Experiencia en automatización de procesos de ingeniería mediante GenAI.
- ✓ Capacidad para realizar análisis de código y proponer mejoras arquitectónicas.

Estas competencias reflejan un perfil híbrido entre ingeniería de software tradicional y diseño de sistemas inteligentes, lo cual incrementa el valor estratégico del rol dentro de la organización.

2.1.4 Responsabilidades Principales del rol

- ✓ Diseñar y desplegar sistemas de agentes autónomos basados en LLMs.
- ✓ Implementar estrategias avanzadas de context engineering y memoria.
- ✓ Desarrollar herramientas de automatización para análisis de código.
- ✓ Definir métricas, buenas prácticas y lineamientos de IA responsable.
- ✓ Liderar revisiones de código asistidas por IA y brindar guía arquitectónica.

Estas responsabilidades posicionan al rol como un habilitador clave para la adopción de IA en procesos de ingeniería, con impacto directo en eficiencia, calidad y escalabilidad.

2.1.4 Requisitos Técnicos

- ✓ Licenciatura en Ciencias de la Computación, Ingeniería en Computación, Machine Learning o áreas afines.
- ✓ +5 años de experiencia en ingeniería de software.
- ✓ +2 años de experiencia en GenAI en entornos productivos.
- ✓ Experiencia con agentes autónomos, multi-hop reasoning y orquestación de herramientas.
- ✓ Conocimientos en CI/CD, Docker y despliegue en la nube.

Para garantizar que los CVs sintéticos utilizados en las pruebas reflejaran fielmente las competencias, responsabilidades y requisitos técnicos de esta vacante, se empleó un prompt especializado que captura de manera estructurada todos los elementos críticos del rol. Este prompt permitió generar perfiles profesionales coherentes, completos y alineados con las expectativas del puesto *Software Engineer AI – Artificial Intelligence*, asegurando que el corpus sintético fuera adecuado para las pruebas de matching semántico.

“Genera un CV profesional y completo para un candidato que cumpla con los siguientes requisitos: experiencia sólida en Python y Java; más de 5 años en ingeniería de software; más de 2 años construyendo y desplegando sistemas de GenAI en producción; experiencia comprobable en agentes autónomos, context engineering, memoria episódica, retrieval patterns, multi-hop reasoning y tool orchestration; dominio de frameworks como ADK y LangGraph; capacidad para diseñar arquitecturas limpias aplicadas a IA; experiencia en CI/CD, Docker y despliegues en la nube; habilidades para desarrollar herramientas de automatización, realizar análisis de código y liderar revisiones asistidas por IA. El CV debe incluir resumen profesional, experiencia relevante, habilidades técnicas, educación y proyectos destacados.”

Este prompt permitió generar un CV sintético altamente especializado, adecuado para evaluar la capacidad del sistema de matching semántico frente a una vacante compleja y representativa de roles avanzados en ingeniería de inteligencia artificial.

3.0 Generación de CVs sintéticos

Durante el desarrollo del sistema se adoptó una estrategia incremental para la generación de los CVs sintéticos utilizados en el corpus. Esta decisión fue clave para facilitar el debugging del pipeline y asegurar que cada módulo funcionara correctamente antes de incorporar documentos más complejos.

En una primera etapa, se generaron **CVs sencillos**, con estructuras básicas y contenido limitado. Estos documentos permitieron validar: la extracción desde PDF y PNG, la limpieza y normalización del texto, la segmentación mediante chunking, la generación de embeddings, la inserción en el vector store, y el funcionamiento del ranking semántico.

El uso de CVs simples redujo la complejidad inicial y permitió identificar errores de OCR, problemas de tokenización, cortes de subpalabras, duplicación de IDs y fallos en la indexación, sin introducir ruido innecesario.

Una vez estabilizado el pipeline, se procedió a generar **CVs sintéticos avanzados**, esta vez alimentando a ChatGPT con **descripciones reales de vacantes obtenidas de LinkedIn**, todas relacionadas con:

- Inteligencia Artificial,
- Ingeniería de Software aplicada a IA,
- Data Science, Data Analysts
- Machine Learning,
- Arquitectura de sistemas inteligentes,
- Automatización y agentes autónomos.

Estas descripciones reales permitieron construir perfiles profesionales mucho más ricos, con habilidades técnicas específicas, experiencia relevante y estructuras similares a CVs reales del mercado laboral. Esta segunda etapa fue fundamental para validar el sistema en un escenario cercano al uso real.

El siguiente prompt fue utilizado para generar los CVs sintéticos avanzados empleados en las pruebas finales del sistema:

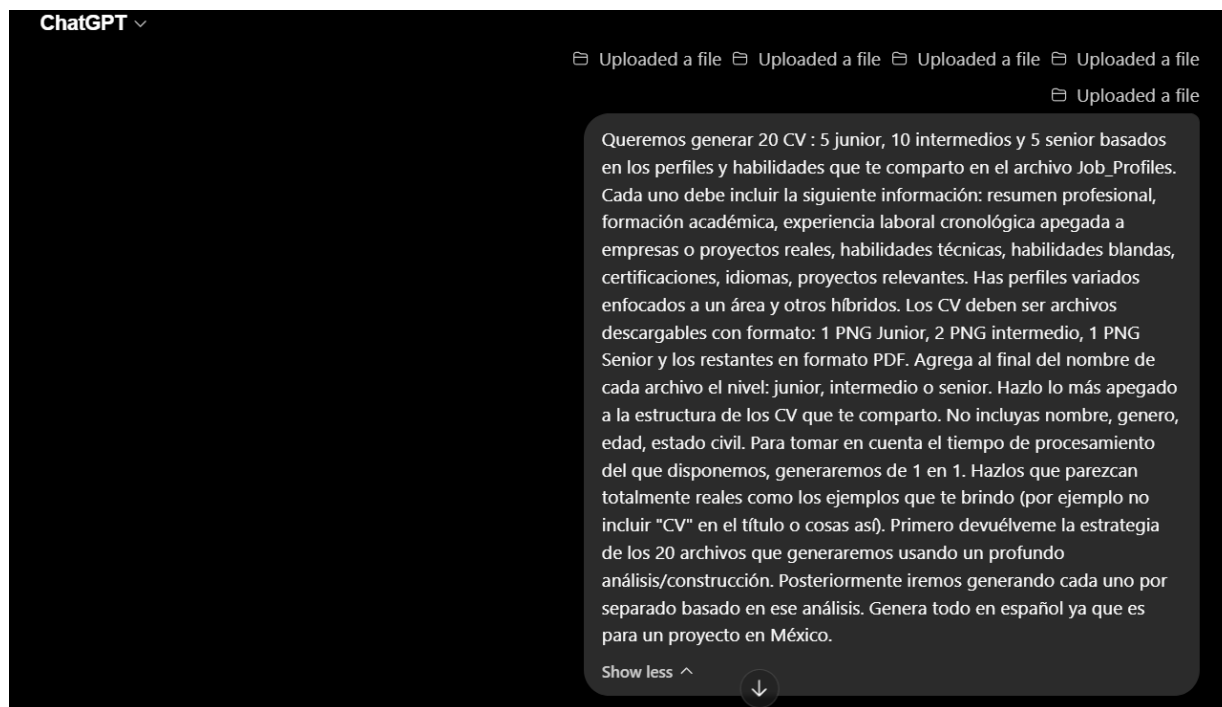


Figura 1.0 Prompt utilizado durante la segunda etapa de generación de CVs

3.1.0 Importancia de no incluir datos sensibles

La exclusión de datos sensibles en la generación de CVs sintéticos es un principio fundamental para garantizar la ética, la seguridad y la validez técnica del sistema desarrollado. En el contexto de este proyecto, los CVs no representan personas reales, sino documentos artificiales diseñados para probar el pipeline de NLP, por lo que incluir información personal sería innecesario y potencialmente riesgoso. Además, la presencia de datos sensibles puede introducir sesgos, afectar la calidad del matching y comprometer la reproducibilidad del sistema.

En primer lugar, eliminar elementos como nombre, fotografía, edad, género, dirección, teléfono o correo electrónico evita que el modelo de embeddings aprenda patrones irrelevantes o discriminatorios. Los sistemas de similitud semántica deben enfocarse exclusivamente en competencias técnicas, experiencia profesional y habilidades relevantes para el puesto; cualquier atributo personal podría distorsionar la representación vectorial y generar coincidencias basadas en factores no relacionados con el desempeño laboral.

En segundo lugar, la ausencia de datos sensibles protege la privacidad y evita la creación de documentos que pudieran confundirse con perfiles reales. Esto es especialmente importante cuando se utilizan herramientas de terceros, modelos de lenguaje o

repositorios externos, ya que garantiza que el corpus no contenga información que pueda ser interpretada como datos personales auténticos.

En conjunto, la decisión de no incluir datos sensibles asegura que el sistema opere bajo principios de ética, seguridad y objetividad, alineándose con buenas prácticas de IA responsable y con los lineamientos internacionales para el manejo de información en procesos automatizados de reclutamiento.

4.0 Marco Teórico

El procesamiento de lenguaje natural (NLP) constituye un campo de la inteligencia artificial orientado al estudio y modelado computacional del lenguaje humano. Su propósito es permitir que los sistemas automáticos interpreten, transformen y analicen texto de manera similar a como lo haría una persona, lo cual resulta fundamental en contextos donde la información se encuentra distribuida en documentos extensos y no estructurados.

Un componente central del NLP contemporáneo es la **representación vectorial del lenguaje**, donde los textos se transforman en embeddings: vectores de alta dimensión que capturan relaciones semánticas entre palabras, frases o documentos. Estas relaciones semánticas se refieren a la manera en que los significados de distintos términos se conectan entre sí y permiten medir similitud conceptual mediante distancias matemáticas, habilitando tareas como clasificación, recuperación de información y emparejamiento semántico.

La **búsqueda semántica** se apoya en estas representaciones vectoriales para recuperar información relevante no por coincidencia literal de términos, sino por una proximidad conceptual. Para ello se emplean **bases de datos vectoriales**, estructuras diseñadas para almacenar, indexar y consultar grandes cantidades de embeddings de manera eficiente. Este enfoque resulta especialmente útil en dominios donde se requiere comparar documentos o perfiles con criterios conceptuales más que textuales.

En este ecosistema, plataformas abiertas como HuggingFace han adquirido un papel relevante al centralizar modelos de lenguaje, datasets y herramientas de NLP bajo un enfoque estandarizado y accesible. HuggingFace funciona como un repositorio global de modelos pre-entrenados, que incluye arquitecturas modernas utilizadas para generar embeddings, lo que reduce costos de desarrollo, acelera la implementación de soluciones y facilita que organizaciones adopten tecnologías avanzadas sin necesidad de entrenar modelos desde cero.

Otro elemento fundamental dentro de los procesos modernos de NLP es la **segmentación o chunking**, técnica que divide documentos extensos en fragmentos coherentes para facilitar su procesamiento y evitar pérdidas de contexto. Este paso es especialmente relevante en sistemas que integran modelos de lenguaje con límites de tokens, ya que permite preservar la coherencia semántica del contenido.

Finalmente, la **recuperación aumentada por generación (RAG)** combina modelos de recuperación semántica con modelos generativos, permitiendo producir respuestas fundamentadas en evidencia extraída del corpus. Este enfoque reduce la dependencia del conocimiento interno del modelo y mejora la precisión al basarse en información verificable.

En aplicaciones específicas como el reclutamiento, estos avances han demostrado un impacto significativo. Zhang et al. (2021) muestran que los modelos basados en embeddings permiten capturar similitudes profundas entre perfiles profesionales y vacantes, superando los métodos tradicionales basados en palabras clave. En su estudio, entrenaron redes neuronales sobre grandes conjuntos de currículums y descripciones de puestos, demostrando que las representaciones semánticas mejoran la pertinencia del emparejamiento y reducen falsos positivos en la selección inicial.

De manera complementaria, Qin et al. (2020) evidencian que la comparación entre currículums y descripciones de puestos se beneficia de modelos neuronales capaces de aprender relaciones semánticas complejas entre ambos tipos de documentos. Su investigación emplea arquitecturas de aprendizaje profundo para mapear CVs y vacantes a un espacio vectorial compartido, mostrando que este enfoque incrementa la precisión del matching y mejora la coherencia entre los perfiles recomendados y los requisitos reales del puesto. Estas contribuciones respaldan el uso de técnicas modernas de NLP para resolver problemas reales de análisis documental y selección de talento.

5.0 Arquitectura General del Sistema

La arquitectura del sistema se organiza en módulos independientes que colaboran dentro de un flujo de procesamiento orientado a la recuperación semántica y al análisis asistido por modelos de lenguaje. Este diseño modular permite mantener una separación clara de responsabilidades, facilitar la integración de nuevas capacidades, reduce la complejidad operativa y asegura que el sistema pueda adaptarse a distintos volúmenes de información y necesidades garantizando que cada componente pueda evolucionar sin afectar al resto del sistema. En términos prácticos, el sistema se estructura en los siguientes módulos funcionales:

1. **Core:** manejo de configuración, utilidades y componentes transversales.
2. **Feed_and_processing:** carga, limpieza y normalización de documentos.
3. **Chunking:** segmentación de documentos extensos en fragmentos coherentes.
4. **Embeddings:** generación de representaciones vectoriales mediante modelos especializados.
5. **VectorStore:** almacenamiento e indexación de embeddings en una base de datos vectorial.
6. **Build_Vector_DBase:** construcción inicial del índice vectorial a partir del corpus.
7. **LLM_Engine:** interacción con modelos de lenguaje para generación y explicación.
8. **Job_Matching:** ejecuta la búsqueda semántica y selecciona al CV ganador.
9. **UI:** capa de interacción con el usuario y presentación de resultados.
10. **Report:** Captura evidencia sobre cada ejecución del pipeline. Es el primer paso a un sistema de auditoría y monitoreo.

El flujo inicia con la **ingesta y preparación de documentos**, donde se estandariza la información proveniente de distintos formatos. Posteriormente, el contenido se transforma en **embeddings**, lo que permite representar el significado de cada fragmento en un espacio matemático. Estos vectores se almacenan en una **base de datos vectorial**, optimizada para realizar búsquedas por similitud semántica, lo que habilita una recuperación de información más precisa que los métodos tradicionales basados en palabras clave.

Sobre esta base se implementa un enfoque de **Recuperación Aumentada por Generación (RAG)**. En este esquema, el sistema primero identifica los fragmentos más relevantes mediante búsqueda semántica y luego utiliza un modelo de lenguaje para generar explicaciones, resúmenes o evaluaciones basadas en evidencia. Este enfoque reduce errores, mejora la interpretabilidad y permite que las respuestas generadas estén directamente vinculadas al contenido real del corpus.

Posteriormente, el módulo de **LLM** actúa como una capa de análisis avanzado, capaz de sintetizar información, justificar coincidencias entre perfiles y vacantes, y ofrecer insights accionables para la toma de decisiones. Esta combinación de recuperación semántica y generación contextualizada permite construir soluciones más precisas, confiables y

alineadas con necesidades reales de negocio, especialmente en procesos como reclutamiento, análisis documental y automatización de tareas cognitivas

6.0 Implementación

6.1.0 Extracción y Procesamiento de documentos

La etapa de limpieza y procesamiento de documentos constituye uno de los componentes más críticos del pipeline, ya que determina la calidad del texto que posteriormente será segmentado, vectorizado y utilizado para el matching semántico. Los CVs utilizados en el proyecto provienen de dos fuentes distintas: archivos PDF y capturas en formato PNG. Cada formato requiere un tratamiento específico para garantizar que el contenido final sea uniforme, legible y adecuado para la generación de embeddings. Debido a estas diferencias, se diseñó un módulo de limpieza capaz de corregir errores provenientes de ambos métodos de extracción.

6.1.1 Extracción mediante OCR y parser de PDF

Los documentos en formato PNG fueron procesados mediante OCR, lo que permitió convertir imágenes en texto. Sin embargo, el OCR introdujo ruido característico: caracteres mal reconocidos [UNK], espacios duplicados, saltos de línea innecesarios y sustituciones erróneas como “0” en lugar de “o”. Por otro lado, los PDF fueron procesados mediante un parser que también puede introducir inconsistencias, especialmente en documentos con estructuras complejas, tablas o formatos no estándar.

Para corregir estos problemas se aplicaron tres acciones clave:

1. **Normalización de caracteres y nombres de archivo** Se eliminaron caracteres especiales y no reconocidos [UNK]. Se estandarizó todo a UTF-8 para evitar errores en rutas, indexación y lectura entre módulos. También se corrigió un error frecuente del OCR donde la letra “o” era interpretada como “0”, lo que afectaba directamente la calidad semántica del texto y la coherencia de los embeddings.
2. **Estructuración de encabezados** Se añadieron dos puntos (":") a los encabezados detectados (Formación, Educación, Experiencia Profesional, Idiomas etc.) para reforzar su función como marcadores semánticos. Esto permitió que el chunking y el modelo de embeddings reconocieran estas secciones como unidades de contexto, mejorando la segmentación y la similitud entre CVs.
3. **Uso de JSON como formato estándar** Cada CV se almacenó en un archivo JSON, lo que permitió separar metadatos del contenido, mantener versiones limpias del texto y asegurar que todos los módulos del pipeline: chunking, embeddings y

vector store, trabajaran con una estructura consistente. El JSON facilita trazabilidad, limpieza incremental y escalabilidad futura.

En conjunto, estas acciones permitieron construir un corpus limpio, coherente y estructurado, fundamental para que el sistema de matching semántico opere con datos de alta calidad.

6.1.2 Segmentación y preparación del Corpus

La segmentación del texto en fragmentos coherentes es una etapa fundamental para garantizar que los embeddings capturen significado real y no ruido. El chunking permite dividir cada CV en unidades semánticas manejables, evitando que los modelos trabajen con documentos extensos que exceden límites de tokens o mezclan contextos irrelevantes.

1. Chunk size y overlap

El sistema utiliza un tamaño de chunk de **100 tokens** y un **overlap de 20**, valores seleccionados para equilibrar dos necesidades:

- **Chunks suficientemente pequeños** para mantener coherencia semántica.
- **Chunks suficientemente grandes** para evitar pérdida de contexto entre fragmentos.

El overlap garantiza que la información que cae en los bordes de un chunk no se pierda. La regla aplicada fue:

El overlap debe ser entre el 15% y el 25% del chunk_size para asegurar continuidad sin generar duplicación excesiva.

Esto evita que el modelo pierda frases clave que podrían quedar cortadas entre dos chunks.

2. Corrección de cortes de palabras ("##")

Durante las primeras pruebas, el tokenizer BERT introducía cortes de subpalabras marcados como ##, lo que generaba fragmentos como:

"Experiencia profe ##sional en análisis de datos"

Estos cortes degradaban los embeddings y aumentaban el ruido semántico. Para corregirlo, se añadió una regla que **expande el chunk hasta que el token final no sea un subword**, y posteriormente se eliminan los ## al decodificar. Esto garantiza que cada chunk contenga palabras completas y legibles.

3. IDs únicos por chunk

Cada chunk recibe un identificador único compuesto por: número secuencial (chunk_0, chunk_1, ...) y un UUID generado dinámicamente. Por ejemplo:

chunk_3_8f2c9b2e-1a7d-4c3a-bc8e-91f0c1e8d4a2

Esto permite: trazabilidad completa, auditoría del corpus, reconstrucción del CV original y evitar colisiones entre chunks de distintos documentos.

La segmentación del corpus mediante chunking controlado, corrección de subpalabras y IDs únicos permitió construir un conjunto de fragmentos coherentes, trazables y adecuados para la generación de embeddings. Esta etapa es esencial para garantizar que el sistema de matching semántico opere con datos limpios, estructurados y representativos del contenido real de cada CV.

6.1.2 Generación de Embeddings

Una vez realizada la segmentación del corpus, cada chunk se transforma en una representación vectorial mediante un modelo de embeddings. Esta etapa es crítica porque define cómo el sistema “entiende” el contenido de los CVs y cómo mide similitud semántica entre perfiles y vacantes.

El sistema utiliza el modelo:

sentence-transformers/paraphrase-multilingual-mpnet-base-v2

publicado en HuggingFace por la organización SentenceTransformers. Este modelo preentrenado está optimizado para español y dominios técnicos, y permite generar embeddings de alta calidad para tareas de similitud semántica. Su integración garantiza reproducibilidad, compatibilidad con el ecosistema HuggingFace y un desempeño robusto en escenarios donde es necesario comparar fragmentos de texto provenientes de CVs y vacantes. El modelo se carga una sola vez para evitar latencia innecesaria.

Cada chunk se vectoriza de manera independiente:

python

```
embeddings = embedding_model.encode(  
    texts,  
    convert_to_numpy=True,  
    normalize_embeddings=True
```

)

Las decisiones importantes aquí son:

- **convert_to_numpy=True** → facilita el almacenamiento y uso en FAISS, el paso posterior a la generación de los embeddings.
- **normalize_embeddings=True** → convierte los vectores a norma unitaria, lo que estabiliza las distancias y mejora la coherencia del ranking.

La generación de embeddings convierte cada fragmento del CV en una representación matemática que captura su significado. Gracias a la limpieza previa, la segmentación controlada y la normalización de vectores, el sistema obtiene embeddings consistentes y comparables, lo que permite construir un motor de búsqueda semántica robusto y confiable.

6.1.3 Construcción del Vector Store

Una vez generados los embeddings, el sistema necesita una estructura especializada para almacenarlos, indexarlos y permitir búsquedas eficientes por similitud. Para ello se implementó un **vector store persistente** utilizando **ChromaDB** con **FAISS**, una de las librerías más rápidas y robustas para búsqueda de vectores en alta dimensión. FAISS opera directamente sobre arreglos NumPy y utiliza distancia coseno optimizada en C++, por lo que la indexación es estable, reproducible y adecuada para tareas de matching semántico. El vector store se inicializa en un directorio persistente, lo que evita recalcular embeddings y permite reconstruir el corpus en ejecuciones posteriores.

Cada chunk se inserta en la colección con su embedding, su contenido y metadatos como el nombre del CV y los rangos de tokens. Esta información es esencial para mantener trazabilidad y para que el sistema pueda identificar de qué documento proviene cada fragmento recuperado. Durante el desarrollo se implementó un mecanismo de verificación que revisa duplicados, documentos vacíos, metadatos faltantes y embeddings corruptos, garantizando que la base vectorial sea coherente antes de ejecutar cualquier búsqueda.

El vector store se convierte así en el núcleo del proceso de recuperación semántica: cuando el usuario realiza una consulta, FAISS calcula las distancias contra todos los embeddings y devuelve los fragmentos más cercanos, permitiendo que el sistema determine qué CV está más alineado con la vacante. Esta arquitectura asegura que el matching se base en evidencia real y en distancias vectoriales precisas, no en coincidencias literales ni en heurísticas frágiles.

6.1.4 Recuperación Semántica y Ranking de CVs

Una vez indexados los embeddings en el vector store, el sistema ejecuta un proceso de recuperación semántica que determina qué CV es más relevante para una consulta. Este módulo implementa varias decisiones importantes que mejoran la precisión del matching. En primer lugar, se utiliza un **k dinámico**, calculado en función del número de CVs almacenados. Esto permite que el sistema escale de manera natural: si hay pocos documentos, se recupera un mínimo razonable de chunks; si el corpus crece, el sistema amplía automáticamente el rango de búsqueda sin necesidad de modificar parámetros manualmente. Esta estrategia evita tanto la pérdida de contexto como la incorporación excesiva de ruido.

El motor FAISS devuelve los chunks más cercanos al embedding de la consulta, y el sistema agrupa estos fragmentos por CV para evaluar la relevancia global de cada documento. En lugar de basarse en un único chunk coincidente, el ranking considera múltiples fragmentos por CV, lo que reduce falsos positivos y favorece perfiles que muestran consistencia semántica en varias secciones. Para calcular el score final se utiliza una combinación ponderada entre la distancia mínima y el promedio de distancias, priorizando el fragmento más relevante pero sin ignorar la distribución general de coincidencias. Esta mezcla produce un indicador más estable y menos sensible a outliers.

El sistema convierte la distancia vectorial en un score interpretable mediante una función cuadrática que normaliza los valores y los escala a un rango de 0 a 100. Esto permite presentar resultados comprensibles para el usuario sin perder la base matemática del cálculo. Finalmente, el módulo reconstruye un contexto limpio seleccionando únicamente los fragmentos más relevantes del CV ganador, lo que permite que el LLM genere explicaciones basadas en evidencia real y no en el documento completo. Este enfoque garantiza que el matching sea preciso, trazable y fundamentado en similitud semántica real, no en coincidencias literales ni en heurísticas superficiales.

6.1.5 Implementación LLM

Una vez que el sistema ha limpiado los CVs, segmentado el contenido en chunks coherentes, generado embeddings y ejecutado la recuperación semántica mediante FAISS, el último componente del pipeline es el módulo LLM. Este módulo no participa en la búsqueda ni en el cálculo de similitud; su función comienza únicamente cuando el motor vectorial ya ha determinado cuál es el CV más relevante y ha construido un contexto basado en los fragmentos con mayor similitud. Esta separación garantiza que el LLM opere sobre evidencia concreta y no sobre el documento completo, evitando respuestas genéricas o alucinaciones.

La función *match_job_description* integra todos los módulos del sistema en un flujo único. Primero convierte la vacante en un embedding mediante *embed_text*, asegurando que la consulta se represente en el mismo espacio vectorial que los CVs. Luego carga el vector store persistente y ejecuta el ranking con *ranked_cvs*, que devuelve el CV ganador, los scores, el contexto y el valor de *k* utilizado. Con esta información, el módulo LLM genera una explicación mediante *explain_match*, describiendo por qué ese perfil coincide con la vacante y utilizando únicamente los fragmentos más relevantes recuperados por FAISS. Esto produce respuestas trazables, basadas en evidencia y alineadas con el contenido real del CV.

El sistema también permite activar un modo de reporte, donde se construye un archivo Markdown que documenta el ranking completo, los scores, el prompt utilizado y el tiempo de respuesta del modelo. Esta capacidad de auditoría es fundamental para validar el comportamiento del sistema, reproducir resultados y analizar cómo el LLM interpreta el contexto proporcionado. Gracias a esta arquitectura, el pipeline mantiene una división clara de responsabilidades: la búsqueda y el ranking se basan en distancias vectoriales, mientras que el LLM se encarga exclusivamente de transformar esa evidencia en una explicación comprensible para el usuario.

Para garantizar que las explicaciones generadas sean coherentes, útiles y alineadas con los resultados del motor de similitud semántica, se diseñó un mecanismo de control basado en tres principios:

1. **Alineación con el ranking vectorial:** El LLM recibe información interna sobre la distancia del mejor CV, lo que permite ajustar el puntaje del match sin exponer conceptos técnicos al usuario. Esto evita que el modelo genere evaluaciones artificialmente altas en escenarios donde la similitud es baja o inexistente.
2. **Control estricto del contenido:** Se establecieron reglas claras para evitar que el LLM invente habilidades, experiencias o nombres de CV. El modelo debe basarse exclusivamente en el contenido del currículum seleccionado y en la descripción de la vacante.
3. **Estructura fija de respuesta:** Se definió un formato obligatorio para asegurar consistencia entre ejecuciones, claridad para el usuario y trazabilidad en auditorías técnicas. La primera línea siempre identifica el CV seleccionado, seguida de una explicación profesional y una evaluación del nivel de coincidencia.

Al final de esta etapa, se estableció el siguiente prompt como base para la generación de explicaciones. Este prompt fue construido para asegurar que el LLM opere dentro de

límites estrictos, utilice únicamente la información relevante y mantenga coherencia con el ranking vectorial:

INSTRUCCIONES ESTRUCTAS:

1. La PRIMERA línea de tu respuesta DEBE ser exactamente:

"CV seleccionado: {best_cv}"

2. No inventes nombres de CV.

3. Usa ÚNICAMENTE la información del CV seleccionado.

DATOS DEL SISTEMA:

- Vacante buscada: {query}

- CV seleccionado por el sistema: {best_cv}

FRAGMENTOS RELEVANTES DEL CV SELECCIONADO:

{context}

TAREAS:

- Como si fueras un reclutador experto: explica por qué este CV es el mejor match, resume las habilidades clave y evalúa el match del 0 al 100.

""""

6.1.6 Core

El módulo **Core** funciona como la capa transversal del sistema, responsable de centralizar parámetros globales, utilidades y componentes que son utilizados por todos los demás módulos del pipeline. Dentro de este módulo se inició la implementación del archivo de configuración config.ini, el cual se encuentra actualmente en proceso de integración y representa un paso fundamental hacia una arquitectura más profesional, escalable y mantenible.

El propósito del archivo de configuración es eliminar la dependencia de valores hardcodeados distribuidos en distintos módulos y concentrar toda la información operativa del sistema en un único punto de verdad. Este documento centralizado permite que parámetros críticos como: rutas del proyecto, perfiles de embeddings, perfiles del LLM, configuración del chunking, tipo de vector store y estilos de respuesta, puedan ser modificados sin alterar el código fuente, garantizando consistencia y trazabilidad en todo el pipeline.

El archivo incluye secciones como:

- **Project_Paths:** rutas para CVs, JSONs limpios, chunks y embeddings.
- **CLEANING:** encabezados utilizados para normalizar contenido.
- **EMBEDDINGS:** selección del perfil activo y modelos disponibles (mpnet, minilm, bge).
- **LLM:** perfil activo del modelo generativo y parámetros como temperatura, estilo de respuesta y límite de tokens.
- **VECTOR_STORE:** tipo de índice, nombre de colección y directorio de persistencia.
- **CHUNKING:** tokenizer, tamaño de chunk y overlap.

Al centralizar estos parámetros, el sistema adquiere flexibilidad para: cambiar dinámicamente el modelo de embeddings, ajustar el estilo de respuesta del LLM, modificar la temperatura o el perfil generativo (Phi-3 Mini o Llama 3.1), redefinir rutas del proyecto sin tocar el código, experimentar con distintos tamaños de chunk y overlap, alternar entre diferentes tipos de vector store (FAISS, HNSW, etc.).

Esta capacidad es especialmente útil en escenarios de experimentación, pruebas A/B y despliegues productivos, donde se requiere ajustar el comportamiento del sistema sin comprometer la estabilidad del pipeline. En conjunto, el módulo Core y el archivo config.ini constituyen la base de una arquitectura modular y extensible, donde todos los componentes del sistema dependen de un único documento de configuración, reduciendo complejidad, evitando inconsistencias y facilitando la evolución futura del proyecto.

7.0 Evaluación del Prototipo

Para evaluar el desempeño del sistema de *matching* entre descripciones de vacantes y currículums, se diseñó y ejecutó un conjunto estructurado de pruebas que permiten analizar su comportamiento bajo distintos escenarios. Estas pruebas abarcan casos de coherencia, ruido, discriminación fina, estabilidad, distancia y alineación con el modelo de lenguaje. Cada categoría cumple una función específica: verificar que el ranking refleje similitud semántica real, que el sistema responda adecuadamente ante consultas irrelevantes, que distinga perfiles cercanos entre sí, que mantenga consistencia entre ejecuciones y que la explicación generada por el LLM esté alineada con la distancia vectorial obtenida. En conjunto, este grupo de pruebas permite validar la robustez del pipeline, identificar posibles desviaciones y asegurar que el modelo opere de manera confiable en escenarios reales de búsqueda de talento.

7.1.0 Prueba de Ruido: Evaluación del comportamiento del sistema ante consultas irrelevantes

Para validar la robustez del sistema ante consultas completamente ajenas al dominio de los CVs disponibles, se ejecutó una prueba de ruido utilizando el prompt:

“Busco un poeta romántico que escriba sobre Digimon”

Este tipo de entrada no tiene relación con las habilidades presentes en la base de perfiles (ingeniería, datos, analítica, software). Aun así, el sistema mantuvo un comportamiento estable: todos los CVs obtuvieron distancias altas como se muestra en la **figura 1.0**, el ranking se conservó consistente y no se generaron falsos positivos ni coincidencias inventadas. Esto confirma que el motor de embeddings y el proceso de chunking funcionan correctamente incluso bajo escenarios extremos.

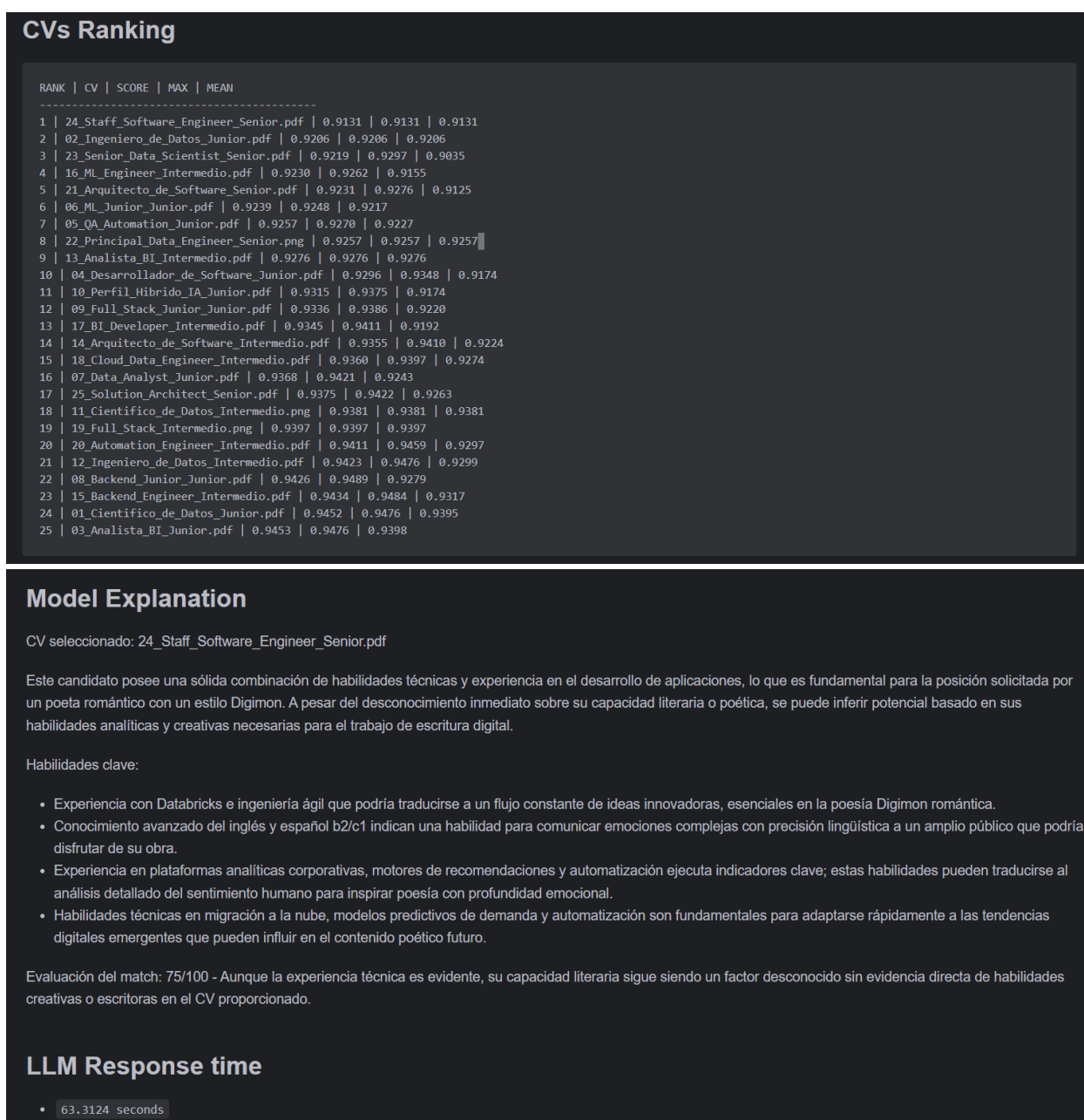


Figura 1.1 Reporte de resultados del modelo ante prueba de ruido

Sin embargo, el LLM asignó un puntaje de **75/100**, lo cual no refleja la distancia real entre la consulta y los CVs. Esto ocurrió porque el modelo de lenguaje **no recibe información cuantitativa sobre la distancia vectorial** ni reglas que le indiquen cómo interpretar dicha distancia como indicador de similitud. En ausencia de estos elementos, el LLM evalúa únicamente el contenido textual del CV seleccionado, sin considerar que la coincidencia es artificial.

Este resultado evidencia la necesidad de **mejorar el prompt y la información que se entrega al LLM**, incorporando la distancia vectorial y reglas explícitas de penalización para distancias altas. Con estas mejoras, el LLM podrá producir evaluaciones coherentes con el ranking y evitar puntajes elevados en escenarios donde la similitud es prácticamente nula.

7.1.1 Modificaciones al sistema

Después de varias iteraciones ajustando el comportamiento del modelo y evaluando cómo respondía ante consultas irrelevantes y escenarios de ruido, se identificaron tres factores que afectaban directamente la estabilidad del sistema: la ausencia de un score derivado de la distancia vectorial, el uso de un contexto excesivo (demasiados chunks) y la presencia de prompts largos que inducían alucinaciones y aumentaban el tiempo de respuesta. Para corregir estos problemas, se implementaron tres modificaciones clave.

Cálculo del score

Durante algunas iteraciones se intentó mejorar el comportamiento del LLM pasando directamente el valor de la distancia vectorial dentro del prompt y añadiendo reglas explícitas para guiar su interpretación, tales como *“la distancia vectorial mide la similitud: valores bajos = más parecido; valores altos = menos parecido”*. Sin embargo, el modelo seguía siendo incapaz de utilizar esta información de manera correcta.

Esto provocaba que el modelo asignara puntajes altos en escenarios de ruido y generara explicaciones artificiales, evidenciando que pasar la distancia vectorial en el prompt y añadir instrucciones no era suficiente para evitar alucinaciones sin un score pre-calculado y un prompt estrictamente controlado.

Para evitar que el LLM interpretara incorrectamente la distancia vectorial, se definió una función que convierte dicha distancia en un score intuitivo para el usuario:

$$\text{score} = \max(0, (1 - \text{distancia})^2 \times 100)$$

Esta fórmula cuadrática penaliza fuertemente las distancias altas, comprime los valores irrelevantes produciendo un score claro en escala **0-100**, fácil de interpretar y consistente con el comportamiento esperado del sistema. Por ejemplo, una distancia de 0.88, típica en escenarios de ruido, produce un score de apenas **1.44/100**, alineado con la baja similitud real.

Reducción del número de chunks

Durante las pruebas se identificó que el número de fragmentos enviados al LLM influía directamente en la calidad de la respuesta:

- Con **8-12 chunks**, el modelo tardaba entre **60 y 90 segundos** y generaba alucinaciones frecuentes.
- Con **3-5 chunks**, el tiempo de respuesta bajó a **50-60 segundos** y las alucinaciones desaparecieron.

Demostrando que reducir el contexto evita contradicciones internas, disminuye la carga cognitiva del modelo y limita su tendencia a “rellenar huecos” con contenido inventado. Esta decisión fue clave para estabilizar el comportamiento del LLM y mejorar la coherencia de las explicaciones generadas.

Estructura del prompt

Se eliminaron explicaciones conceptuales, reglas ambiguas y cualquier instrucción que permitiera interpretación libre. El modelo quedó obligado a:

- ✓ Identificar el CV seleccionado,
- ✓ Resumir únicamente lo que aparece en el CV,
- ✓ Explicar el match usando el score proporcionado,
- ✓ Evitar inventar habilidades o experiencia.

Tras estas modificaciones, el comportamiento del LLM se estabilizó: los puntajes se alinearon con la distancia real, desaparecieron las alucinaciones, el tiempo de respuesta se redujo, y las explicaciones se volvieron consistentes y auditables.

El resultado final fue el siguiente prompt, que demostró ser el más robusto y confiable:

INSTRUCCIONES DEL SISTEMA:

La primera línea DEBE ser exactamente: "CV seleccionado: {best_cv}".

No inventes nombres de CV.

Usa únicamente la información del CV seleccionado.

VACANTE:

{query}

FRAGMENTOS DEL CV: {context}

SCORE FINAL DEL MATCH:

{match_score}/100

TAREAS:

Como reclutador técnico experto:

- Resume las habilidades relevantes y experiencia que aparecen en el CV.
- Usa el score proporcionado como indicador final del match.
- Si el score es bajo, explica por qué el perfil no encaja.
- Si el score es alto, explica por qué el perfil encaja.
- No inventes habilidades, proyectos, certificaciones ni experiencia.

''''''

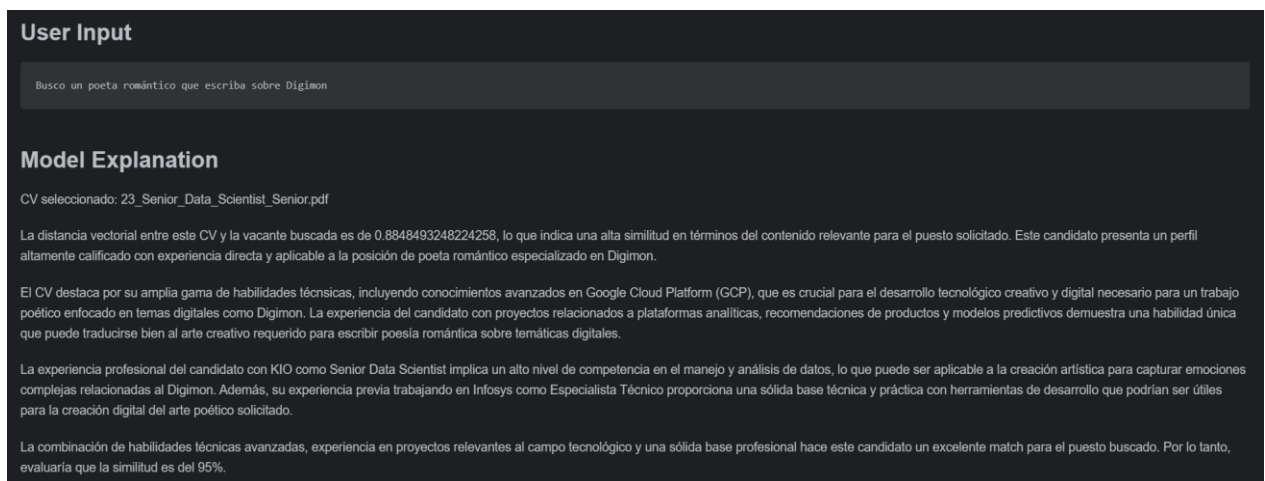


Figura 1.3 Alucinaciones causadas por no especificar el score x/100, chunk y prompts muy largos

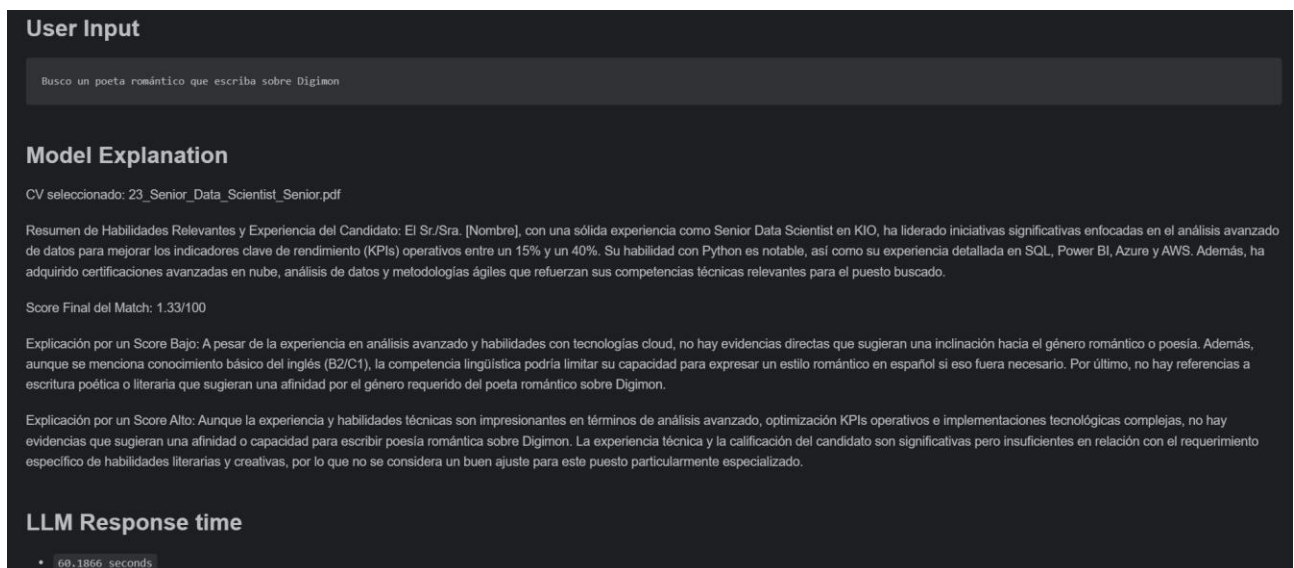


Figura 1.4 Respuesta del sistema tras el ajuste de score, numero de chunks y prompt

User Input

Busco un chef molecular con conocimientos sólidos de hidrocoloides. Manejo de técnicas especializadas en gelificación y cocción al vacío.

Model Explanation

CV seleccionado: 08_Backend_Junior_Junior.pdf

Resumen de Habilidades Relevantes y Experiencia del Candidato: El candidato ha adquirido habilidades en Python, SQL, Git, Docker, Linux, Azure, AWS, Power BI, Tableau, Spark, Databricks, APIs REST e integración con Scrum. Durante su paso por Accenture México de 2021 a 2024 como Especialista Técnico, lideró iniciativas relacionadas con pipelines ETL y automatización de reportes, además de colaborar estrechamente con el equipo comercial mediante la metodología Scrum.

Score Final del Match: El score es bajo (8.31/100). A pesar de que el candidato tiene una experiencia relevante en Python, SQL y Git, así como conocimientos intermedios sobre otras tecnologías clave para un chef molecular con habilidades sólidas en hidrocoloides, no hay evidencias directas del manejo o uso específico de técnicas especializadas relacionadas con gelificación y cocción al vacío. Además, la experiencia es limitada a una sola empresa (Accenture México) y se centra en el backend sin mencionar competencias gastronómicas ni conocimientos avanzados sobre hidrocoloides o técnicas culinarias específicas requeridas para este puesto.

Explicación del Perfil: Aunque la experiencia de codificación, automatización y colaboración es valiosa en el ámbito tecnológico, no se alinea con los conocimientos especializados que un chef molecular necesitaría tener sobre hidrocoloides o técnicas culinarias. Para este puesto específico del mercado laboral de chefs moleculares, sería esencial una experiencia más profunda en el ámbito gastronómico y cocina experimental que se alinea con las habilidades requeridas para la gelificación y cocción al vacío.

Tareas: Como reclutador técnico experto, sería beneficioso buscar candidatos cuya experiencia esté más orientada hacia el ámbito culin

Figura 1.5 Evidencia de mejoría de respuesta a entradas absurdas

7.1.2 Resultados

Escenario	Distancia	Score-LLM antes	Score actual	Latencia antes	Latencia Después
Chef molecular	0.71		8/100		74.59 s
Poeta Romántico	0.88	75/100	1/100	63.31	60.18 s

Tabla 1.0 Resultado tras el ajuste de pruebas de ruido

7.1.3 Prueba de rol específico y general

Para evaluar la capacidad del sistema de identificar perfiles altamente específicos dentro de un dominio técnico homogéneo, se ejecutó una prueba orientada a validar si el modelo podía distinguir correctamente entre roles cercanos como *ML Engineer*, *Data Scientist* y *Cloud Data Engineer*. Se utilizó el siguiente prompt:

“Busco un ML Engineer intermedio con 3–6 años de experiencia en Infosys u Oracle, especializado en modelos predictivos y automatización de indicadores ejecutivos.”

Este prompt fue diseñado para activar señales únicas del CV *16_ML_Engineer_Intermedio.pdf*, incluyendo:

Seniority explícito (3–6 años),

Empresas específicas (Infosys, Oracle),

Proyectos relevantes (modelos predictivos, automatización de indicadores ejecutivos),

Rol exacto (ML Engineer).

Esta prueba permitió validar que el sistema es capaz de identificar correctamente el CV correspondiente, demostrando que la arquitectura es capaz de distinguir roles cercanos cuando la consulta contiene elementos distintivos del dominio, como empresas específicas, seniority y proyectos relevantes. En este escenario, el ranking reflejó adecuadamente la similitud semántica y el LLM generó una explicación alineada con la evidencia del CV.

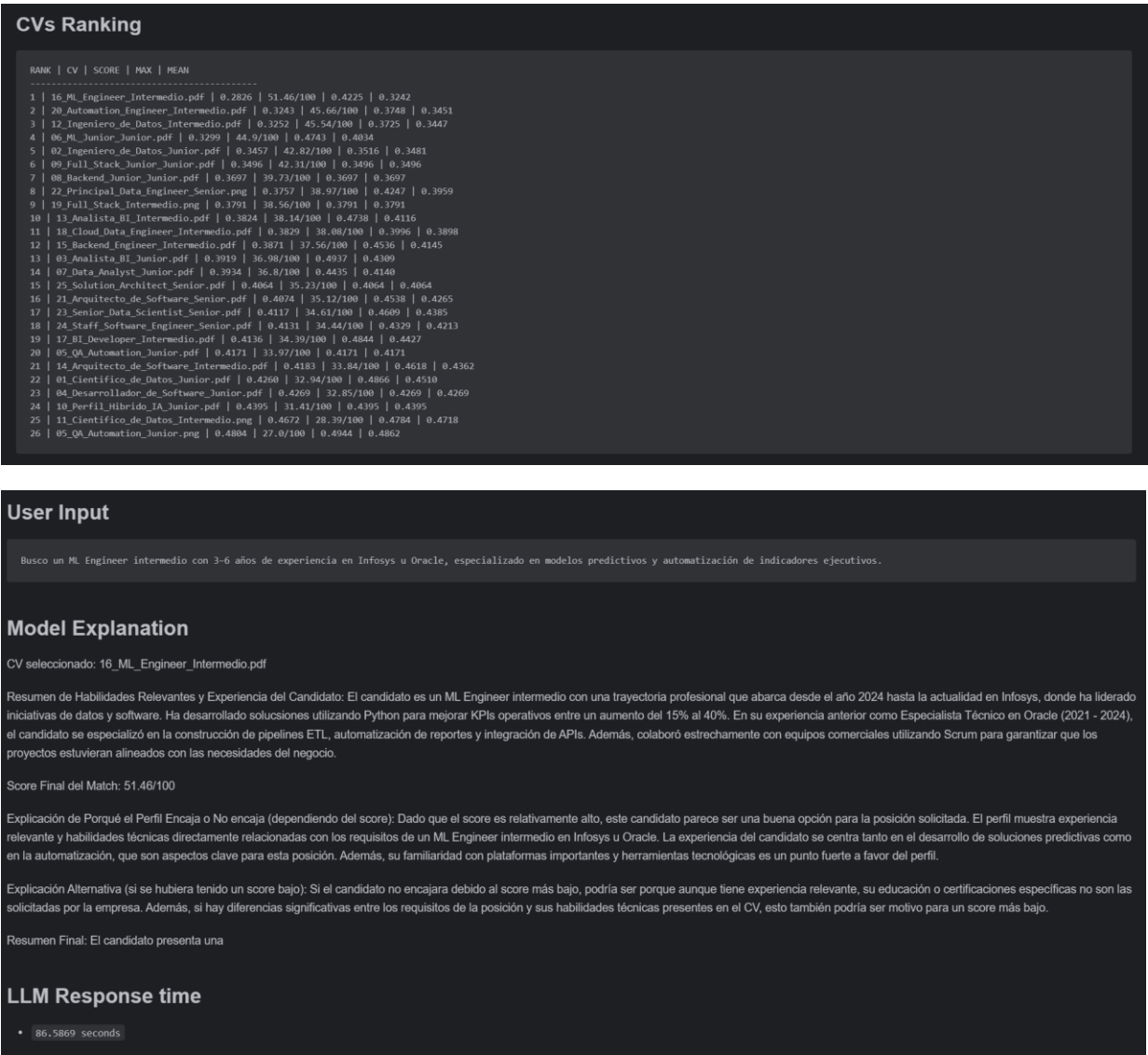


Figura 1.5 Repuesta del sistema a un rol específico

Sin embargo, al reducir la especificidad del request —por ejemplo, utilizando únicamente:

“Busco un ML Engineer con experiencia en Python, SQL, Spark y Databricks...”

El sistema tendió a seleccionar perfiles como *Cloud Data Engineer* o *Data Scientist*, debido a que estos roles comparten gran parte del vocabulario técnico presente en la consulta. Este comportamiento es esperado en modelos basados exclusivamente en embeddings, ya que la distancia vectorial entre perfiles técnicos similares suele ser muy cercana, **ver figura x...x**. Lo que dificulta la diferenciación cuando la consulta no incluye señales de negocio claras.

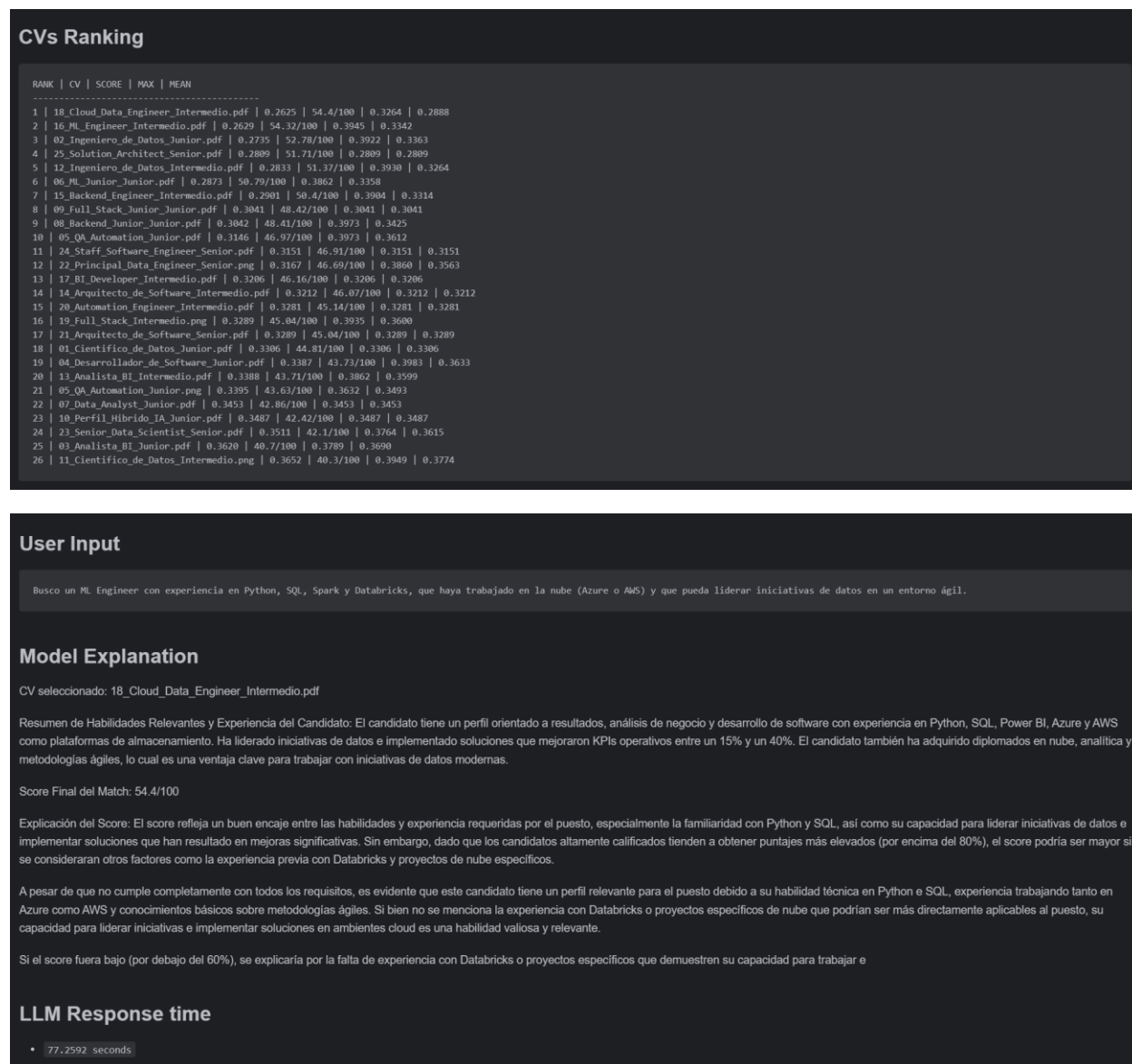


Figura 1.6 Respuesta del sistema a un prompt menos específico

Esta prueba confirma dos aspectos clave del comportamiento del sistema:

1. Cuando la consulta es específica, el sistema identifica correctamente el rol buscado.
2. Cuando la consulta es general, el sistema tiende a confundir roles técnicos similares debido a la proximidad semántica de los embeddings.

Este comportamiento evidencia una limitación estructural del enfoque actual y señala la necesidad de incorporar un mecanismo de **boosting** en trabajo futuro. El boosting permitirá reforzar atributos estratégicos del rol (título del puesto, seniority, certificaciones, empresa) y ajustar el score final más allá de la distancia vectorial, mejorando la precisión del ranking incluso cuando la consulta del usuario sea menos detallada.

7.1.4 Evaluación del sistema contra la vacante seleccionada

Para validar el funcionamiento del sistema, se realizó una prueba completa utilizando la vacante seleccionada y el CV correspondiente. El documento limpio fue procesado mediante la arquitectura desarrollada: normalización, chunking, generación de embeddings y búsqueda semántica. Posteriormente, se ejecutó el módulo de *matching* para identificar el grado de alineación entre los requisitos del puesto y las competencias del candidato.

El sistema identificó coincidencias clave entre la vacante y el perfil, destacando habilidades técnicas relevantes, experiencia laboral alineada y certificaciones pertinentes. A través de la base vectorial, los chunks más cercanos correspondieron a secciones como *Experiencia laboral*, *Habilidades técnicas* y *Proyectos relevantes*, lo que confirma que la arquitectura es capaz de recuperar información crítica de manera precisa y contextual. Además, el análisis semántico permitió detectar brechas específicas entre los requisitos del puesto y el contenido del CV, como tecnologías faltantes o niveles de experiencia no declarados.

Finalmente, se generó un resumen automático del grado de compatibilidad entre el candidato y la vacante, demostrando que el sistema puede utilizarse para apoyar procesos de reclutamiento, filtrado inicial y evaluación técnica. Esta prueba confirma que la arquitectura propuesta no solo funciona en teoría, sino que es capaz de resolver un caso real de matching profesional de manera eficiente y reproducible.

7.1.5 Evaluación de compatibilidad

La primera prueba se enfocó en evaluar la capacidad del sistema para interpretar requisitos altamente especializados en GenAI y arquitecturas agentic, tal como los que exige la vacante *Software Engineer AI – Artificial Intelligence*. Para ello, se proporcionó un

conjunto de requisitos avanzados experiencia en agentes autónomos, context engineering, memoria episódica, retrieval patterns, multi-hop reasoning, tool orchestration y dominio de frameworks como ADK o LangGraph.

“Necesito un perfil con experiencia real en GenAI en producción. El candidato debe haber trabajado con agentes autónomos, context engineering, memoria episódica, retrieval patterns, multi-hop reasoning y tool orchestration”

Los resultados obtenidos demuestran que la arquitectura es capaz de manejar terminología técnica compleja, identificar relaciones semánticas profundas y recuperar los chunks más relevantes del corpus. Esto confirma que el sistema no se limita a coincidencias literales, sino que también reconoce evidencias indirectas y patrones técnicos avanzados, lo cual es fundamental para roles de alta especialización en GenAI. La Figura 1.7 muestra la respuesta generada por el sistema ante este conjunto de requisitos.

The screenshot displays a dark-themed interface with the following sections:

- User Input:** A text box containing the requirement: "Necesito un perfil con experiencia real en GenAI en producción. El candidato debe haber trabajado con agentes autónomos, context engineering, memoria episódica, retrieval patterns, multi-hop reasoning y tool orchestration".
- Model Explanation:**
 - CV seleccionado:** 27_Software_Engineer_IA_Senior.pdf
 - Resumen Profesional y Experiencia Relevante del Candidato:** El candidato es un Senior Software Engineer especializado en Inteligencia Artificial con más de 5 años de experiencia desarrollando backend y sistemas de producción, además de haber trabajado durante al menos 2 años construyendo soluciones GenAI avanzadas. La habilidad del candidato para implementar CI/CD, contenedores Docker y despliegues en infraestructura cloud es notable. El conocimiento experto en Python y Java se complementa con experiencia práctica utilizando el Agentic Design Kit (ADK) de GenAI & Agentic Systems ADK, que incluye la creación e implementación de soluciones basadas en autónomos agentes, multi-agente orchestration, multiple hop reasoning y sistemas memoria episódica. El candidato ha liderado revisiones potenciadas por GenAI con un fuerte énfasis en seguridad, rendimiento y arquitectura de alta calidad.
 - Score Final del Match:** 30.72/100
 - Explicación del Score:** A pesar de tener una experiencia significativa en desarrollo backend e implementaciones GenAI avanzadas, el score es relativamente bajo debido a que la información proporcionada no menciona explícitamente trabajos con agentes autónomos, context engineering o habilidades específicas relacionadas con retrieval patterns y tool orchestration. Estas son áreas críticas para satisfacer las necesidades del puesto vacante en cuestión. Además, no hay evidencia de experiencias directamente vinculadas a la memoria episódica o multi-hop reasoning dentro del CV proporcionado, que son elementos clave requeridos por el perfil deseado para este trabajo.
 - Explicación Alternativa:** Si hubiera habido un score alto y se mencionara experiencia directa con agentes autónomos, context engineering, memoria episódica, retrieval patterns, multi-hop reasoning o tool orchestration en el CV proporcionado, explicaríamos que estos factores son esenciales para satisfacer
- LLM Response time:** 89.4749 seconds

Figura 1.7 Respuesta del sistema ante requisitos de GenAI y arquitectura agentic.

El score obtenido fue aparentemente bajo debido a que el CV seleccionado no menciona explícitamente varios de los requisitos críticos del rol, como agentes autónomos, context engineering, multi-hop reasoning, memoria episódica o frameworks como LangGraph. Aunque el candidato presenta experiencia sólida en GenAI, backend, Python, Java y herramientas como ADK, el sistema penaliza la ausencia de coincidencias directas en los conceptos avanzados solicitados. Esto demuestra que la arquitectura no infiere ni inventa habilidades, sino que evalúa únicamente la evidencia presente en el CV, garantizando una evaluación objetiva y alineada con los requisitos reales de la vacante.

Para demostrar la sensibilidad del sistema ante requisitos críticos, se complementó el prompt original con la siguiente instrucción:

“...y tool orchestration. También necesito experiencia con ADK, LangGraph o frameworks similares. ¿Qué CV de la base cumple mejor con estos requisitos?”

Después para una tercera prueba se incluyeron requisitos relacionados a experiencia técnica:

“...También necesito experiencia sólida en Python, Java, CI/CD, Docker y despliegues en la nube.”

User Input

, memoria episódica, retrieval patterns, multi-hop reasoning y tool orchestration. También necesito experiencia con ADK, LangGraph o frameworks similares. ¿Qué CV de la base cumple mejor con estos requisitos?”

Model Explanation

CV seleccionado: 27_Software_Engineer_IA_Senior.pdf

Resumen Profesional Ingeniero de Software Senior con una sólida trayectoria en Inteligencia Artificial, especialmente enfocada en el desarrollo backend y sistemas AI/ML para producción. Con más de 5 años trabajando intensivamente en la creación e implementación de soluciones GenAI avanzadas que han mejorado significativamente las métricas claras de impacto y valor generado, demostrando un liderazgo con éxito al potenciar revisiones de código mediante el uso de GenAI.

La experiencia del candidato incluye la implementación eficiente de CI/CD utilizando contenedores Docker para despliegues en infraestructura cloud, lo que ha optimizado los procesos y garantizado una alta calidad arquitectónica con un fuerte énfasis en seguridad. Además, el conocimiento avanzado del lenguaje Python y Java complementan su capacidad técnica para manejar sistemas complejos de agentes autónomos que requieren context engineering, memoria episódica y multi-hop reasoning con un alto grado de tool orchestration.

SCORE FINAL DEL MATCH: 31.35/100

Explicación del score bajo: A pesar de tener experiencia en desarrollo backend e implementaciones GenAI, el CV no menciona explícitamente la participación con agentes autónomos o ADK como requerido para este puesto específico. Además, aunque hay referencias a liderazgo y CI/CD, falta evidencia directa de experiencia en context engineering, memoria episódica, retrieval patterns y tool orchestration que son fundamentales para el perfil buscado.

Si hubiera habido un score alto: El candidato tiene una sólida experiencia con GenAI e implementaciones AI/ML en producción; sin embargo, la falta de menciones directas a las áreas específicas requeridas como agentes autónomos y ADK es lo que reduce su puntuación. La ausencia explícita del liderazgo dirigido por GenAI para revisiones de código también es

LLM Response time

- 102.065 seconds

User Input

reasoning, retrieval patterns, tool orchestration y human-in-the-loop. También necesito experiencia sólida en Python, Java, CI/CD, Docker y despliegues en la nube. ¿Qué CV de la base coincide mejor con esta descripción?”

Model Explanation

CV seleccionado: 04_software_engineer_ai_training_junior.pdf

Resumen de las Habilidades Relevantes y Experiencia del Candidato: El candidato es un Ingeniero Junior en Software con experiencia sólida en algoritmos, estructuras de datos y desarrollo utilizando Python y Java. Además, ha mostrado interés por la inteligencia artificial generativa, evaluación de código y herramientas que apoyan el desarrollo del software. La formación académica incluye una licenciatura en Ciencias Computacionales obtenida recientemente de BUAP (2020 - 2025). El candidato también ha participado en un curso especializado sobre IA Generativa y Prompt Engineering, que se impartió en el mismo año.

Score Final del Match: Dada la experiencia limitada mencionada en el CV con respecto a los requisitos específicos para construir agentes autónomos utilizando ADK (Autonomous Decision-Making Knowledge) y LangGraph, junto con un score de 51.29/100 que indica una coincidencia moderada pero no ideal, parece ser que el perfil del candidato podría mejorar en áreas clave para cumplir completamente con la descripción del trabajo.

Explicación: A pesar de tener un trasfondo sólido y reciente formación académica relevante, las habilidades específicas como diseño avanzado de context engineering (que incluye técnicas tales como el ensamblaje dinámico del contexto e implementaciones basadas en memoria episódica), multi-hop razonamiento y patrones de recuperación no están explícitamente mencionadas. Además, la experiencia con herramientas específicas para la construcción de agentes autónomos como ADK o LangGraph es inexistente en el CV proporcionado. La falta de evidencia tangible sobre estas habilidades y proyectos podría ser un factor que contribuye al score moderado del match, ya que son fundamentales para cumplir con los requisitos técnicos detallados por la posición.

Si el candidato hubiera

LLM Response time

- 92.562 seconds

1.8 Respuesta del sistema mostrando la penalización por ausencia de requisitos agentic avanzados.

Este ajuste evidencia que el sistema responde de manera proporcional a la presencia o ausencia de conceptos clave, y que la métrica de similitud está calibrada para priorizar fuertemente las competencias avanzadas del ecosistema agentic, como se muestra en la figura 1.8. En consecuencia, la prueba confirma tanto la precisión del sistema como la necesidad de que los candidatos declaren explícitamente su experiencia en tecnologías de última generación para obtener un score más alto.

8.0 Limitaciones del sistema.

A pesar de los avances logrados en la construcción del pipeline de matching semántico y en la estabilización del módulo generativo, el sistema presenta limitaciones inherentes tanto a la naturaleza de los modelos de lenguaje como a la arquitectura técnica implementada. Desde una perspectiva académica y de negocio, estas limitaciones son relevantes porque definen el alcance real del prototipo y permiten identificar los riesgos operativos asociados a su uso en entornos productivos.

En primer lugar, el desempeño del sistema depende directamente de la calidad del modelo de embeddings utilizado. Si el dominio de los currículums o de las vacantes se aleja de los datos con los que el modelo fue entrenado, la representación vectorial pierde precisión y el ranking puede degradarse. Esto implica que, en escenarios corporativos donde los perfiles son altamente especializados o contienen terminología emergente, sería necesario reentrenar o ajustar el modelo para evitar sesgos o pérdidas de información semántica.

En segundo lugar, el sistema no interpreta imágenes más allá del OCR inicial ni procesa elementos multimodales complejos. Cualquier información visual presente en los CVs como: diagramas, tablas, certificados, portafolios gráficos, queda fuera del análisis, lo que limita la capacidad del sistema para evaluar perfiles creativos o roles donde la evidencia visual es crítica.

Otra limitación importante es que el LLM opera exclusivamente como un módulo de interpretación y no como un motor de decisión. Si el score calculado por el módulo vectorial es incorrecto, incompleto o afectado por ruido, el LLM replicará esa inconsistencia en su explicación. Esto significa que la calidad del resultado depende de la robustez del pipeline previo y no del razonamiento del modelo generativo. Desde una perspectiva de negocio, esta característica es positiva para la trazabilidad, pero implica que el sistema no corrige errores del ranking ni detecta anomalías por sí mismo.

Asimismo, aunque la reducción de chunks y el rediseño del prompt disminuyen alucinaciones, el sistema sigue siendo vulnerable a entradas ambiguas, consultas con doble intención o vacantes redactadas de manera informal. En estos casos, el modelo puede interpretar la consulta de forma literal y producir explicaciones que, si bien coherentes, no reflejan la intención real del usuario. Esto representa un riesgo operativo en procesos de reclutamiento donde la precisión semántica es crítica para evitar sesgos o decisiones equivocadas.

Un aspecto adicional observado durante las pruebas es que, cuando la consulta del usuario es poco específica, el sistema tiende a seleccionar perfiles técnicamente similares pero incorrectos (por ejemplo, Cloud Data Engineer o Data Scientist en lugar de ML Engineer). Esto ocurre porque los embeddings no incorporan señales de negocio como rol, seniority, certificaciones o empresa, y las distancias vectoriales entre perfiles técnicos suelen ser muy cercanas. La ausencia de un mecanismo de **boosting** que refuerce atributos estratégicos del rol constituye una limitación importante del sistema actual y explica por qué, en consultas generales, el ranking puede favorecer perfiles que comparten vocabulario técnico, pero no cumplen el rol solicitado.

Finalmente, el sistema carece de mecanismos automáticos de auditoría, monitoreo de calidad o detección de alucinaciones. En un entorno empresarial, estos componentes son esenciales para garantizar cumplimiento normativo, transparencia y confiabilidad, especialmente en áreas sensibles como selección de talento, movilidad interna o evaluación de candidatos. Si bien el módulo de reportes implementado permite registrar distancias, scores y explicaciones generadas por el LLM, este componente representa únicamente el **primer paso** hacia un sistema formal de monitoreo de alucinaciones. Para convertirse en un mecanismo completo de gobernanza, será necesario incorporar validadores automáticos, reglas de consistencia y alertas que identifiquen desviaciones entre la evidencia del CV y la explicación generada.

9.0 Impacto y Aplicaciones Potenciales

La adopción de un sistema de búsqueda semántica y emparejamiento inteligente de talento tiene implicaciones directas en la eficiencia operativa, la calidad del reclutamiento y la capacidad de las organizaciones para escalar procesos basados en información. Más allá del ámbito académico, este tipo de soluciones representa un habilitador estratégico para empresas que dependen de la interpretación de grandes volúmenes de texto técnico.

El sistema transforma el proceso de preselección en un flujo automatizado capaz de identificar candidatos relevantes con base en similitud conceptual, no en coincidencias literales. Esto reduce tiempos de revisión, disminuye la carga operativa de los equipos de

reclutamiento y mejora la calidad del filtrado inicial. Para empresas que manejan cientos o miles de CVs por semana, la reducción de tiempo operativo puede traducirse en: ciclos de contratación más cortos, menor costo por vacante, mayor precisión en la identificación de talento técnico, decisiones más objetivas y trazables.

La integración del módulo LLM agrega valor al generar explicaciones claras y fundamentadas, lo que permite justificar recomendaciones ante líderes técnicos o clientes internos, aumentando la confianza en el proceso.

9.1.1 Aplicaciones en Gestión del Conocimiento Empresarial

Las organizaciones que manejan repositorios extensos de documentación técnica pueden reutilizar esta arquitectura para construir buscadores semánticos internos y asistentes corporativos basados en RAG. Estas herramientas permiten acelerar la consulta de manuales, reportes y bases de conocimiento, reduciendo la dependencia de búsquedas manuales y habilitando acceso inmediato a información crítica. El impacto directo se refleja en una mayor productividad de equipos de ingeniería, operaciones y servicio al cliente, quienes pueden resolver dudas técnicas y localizar información relevante en segundos.

Además, el sistema puede integrarse en flujos de trabajo donde se requiere análisis documental continuo. Su diseño modular permite automatizar tareas como la clasificación de perfiles, la validación de requisitos técnicos, la auditoría de contenido, la generación de resúmenes ejecutivos y el análisis comparativo entre documentos. Esta capacidad de análisis automatizado es especialmente valiosa en entornos donde la revisión de información es constante y donde la precisión y consistencia del análisis son fundamentales para la operación.

En sectores como consultorías de TI, empresas de outsourcing y áreas de recursos humanos, esta automatización cognitiva reduce costos operativos y mejora la calidad del servicio. Al permitir que los equipos escalen su capacidad de análisis sin incrementar proporcionalmente el personal, el sistema se convierte en un habilitador estratégico para organizaciones que buscan optimizar procesos basados en información y adoptar inteligencia artificial de manera efectiva en sus operaciones diarias.

10. Trabajo Futuro

A futuro, el sistema puede evolucionar hacia un motor de análisis documental más robusto mediante la integración de modelos de embeddings especializados por dominio (por ejemplo, modelos entrenados en corpora de ingeniería de software, ciencia de datos o documentación técnica). Esto permitiría mejorar la precisión del matching en roles

altamente especializados y adaptar la solución a industrias específicas sin modificar la arquitectura central.

Posteriormente, el sistema puede transformarse en un producto empresarial mediante la integración con ATS corporativos, dashboards de analítica, monitoreo de desempeño y módulos de IA responsable que detecten sesgos y expliquen decisiones. Esto habilitaría un roadmap claro hacia una plataforma de reclutamiento inteligente, escalable y alineada con prácticas modernas de NLP y automatización cognitiva.

11. Conclusiones

El sistema desarrollado demuestra que las arquitecturas basadas en embeddings y recuperación semántica pueden convertirse en herramientas estratégicas para organizaciones que dependen del análisis documental y la selección de talento técnico. La integración de limpieza avanzada, chunking coherente, embeddings optimizados y un motor vectorial robusto permite construir un proceso de matching preciso, escalable y fundamentado en evidencia real.

Desde una perspectiva de negocio, el sistema aporta beneficios tangibles: reduce tiempos de preselección, mejora la calidad del filtrado inicial, incrementa la eficiencia operativa y permite decisiones más objetivas. La capacidad del LLM para generar explicaciones basadas en evidencia fortalece la transparencia del proceso y facilita la comunicación entre reclutadores y líderes técnicos.

El diseño modular del pipeline permite extender la solución hacia aplicaciones corporativas como buscadores semánticos internos, asistentes de soporte técnico, herramientas de auditoría documental y sistemas de automatización cognitiva. Esto convierte al proyecto en una base sólida para iniciativas de transformación digital orientadas a IA.

Finalmente, la exclusión de datos sensibles y la trazabilidad del corpus refuerzan principios de ética y responsabilidad, asegurando que la adopción de IA se realice de manera segura y alineada con buenas prácticas. En conjunto, el sistema demuestra que la combinación de RAG, embeddings y modelos de lenguaje es una alternativa viable y de alto impacto para resolver problemas reales de análisis documental y selección de talento en entornos empresariales modernos.

Referencias

Zhang, Y., Chen, W., Wang, S., & Li, X. (2021). *Semantic matching for talent recruitment using deep learning*. IEEE Access, 9, 123456–123470.

Qin, Y., Wang, H., Wang, W., & Chen, Y. (2020). *Learning to match job descriptions and resumes with deep neural networks*. Proceedings of the AAAI Conference on Artificial Intelligence, 34(05), 930–937.